

RELATÓRIO TÉCNICO

Assistente Médico com LLM, Fine-tuning e LangChain

1. Introdução

Este projeto apresenta o desenvolvimento de um protótipo de assistente médico baseado em Large Language Model (LLM), combinando fine-tuning com LoRA, LangChain, Retrieval-Augmented Generation (RAG) e banco vetorial Chroma.

O objetivo é desenvolver um sistema capaz de receber perguntas relacionadas à área médica, recuperar informações relevantes de uma base de conhecimento e gerar respostas contextualizadas.

A solução foi desenvolvida como um protótipo acadêmico, utilizando bases públicas de dados médicos como alternativa aos dados internos de um hospital. Dessa forma, os resultados demonstram a viabilidade técnica da arquitetura, mas não representam um sistema clínico pronto para utilização em ambiente hospitalar.

2. Dados utilizados

Foram utilizadas duas bases públicas com finalidades complementares: **PubMedQA**, utilizada na preparação e adaptação do modelo, e **MedQuAD**, utilizada como base de conhecimento para o sistema de recuperação.

2.1 PubMedQA

O PubMedQA é um conjunto de dados composto por perguntas médicas acompanhadas de contextos científicos e respostas. A base utilizada possui **1.000 registros**, contendo campos como pergunta, contexto, resposta, decisão e ano.

A distribuição das decisões foi:

Decisão	Quantidade	Percentual
Yes	552	55,2%
No	338	33,8%
Maybe	110	11,0%
Total	1.000	100%

As perguntas possuem média de aproximadamente **94,20 caracteres e 12,91 palavras**, enquanto as respostas longas possuem média de aproximadamente **268,37 caracteres e 39,66 palavras**.

A base apresentou ausência de perguntas, respostas e contextos vazios, além de nenhuma pergunta ou resposta duplicada.

Também foi realizada uma verificação preliminar de possíveis informações identificáveis. Não foram encontrados e-mails. As ocorrências numéricas estavam principalmente associadas a anos e referências, além de uma URL relacionada à fonte científica. Verificação realizada para garantir que não havia dados que demandavam anonimização (conforme LGPD).

2.2 MedQuAD

O MedQuAD foi utilizado como base de conhecimento para o componente de recuperação do assistente.

Foram processados **981 arquivos XML**, contendo pares de perguntas e respostas médicas. O conjunto resultou em um DataFrame com os campos:

- qid
- qtype
- focus
- question
- answer

Todos os registros apresentaram o tipo de pergunta `information`.

A média das perguntas foi de aproximadamente **36,13 caracteres**, enquanto as respostas apresentaram média de aproximadamente **896,33 caracteres**.

Também foram verificadas duplicidades, não sendo identificadas perguntas ou respostas duplicadas.

A utilização foi feita com base no critério `focus`, o que melhor se adaptou para trazer um resultado consistente na formatação da resposta.

3. Pré-processamento e divisão dos dados

Os dados foram submetidos a etapas de organização, validação e preparação para utilização no pipeline.

No caso do PubMedQA, os dados foram separados de forma estratificada em:

- **70% para treinamento**
- **20% para validação**
- **10% para teste**

A divisão manteve aproximadamente a mesma proporção das classes.

Conjunto	Yes	No	Maybe
Treinamento	55,1%	33,9%	11,0%
Validação	55,5%	33,5%	11,0%
Teste	55,0%	34,0%	11,0%

A estratificação foi utilizada para evitar alterações significativas na distribuição das classes entre os diferentes conjuntos.

4. Fine-tuning do modelo

O processo de adaptação foi realizado utilizando uma LLM com aproximadamente **1,55 bilhão de parâmetros**.

Para reduzir o custo computacional, foi utilizada a técnica **LoRA (Low-Rank Adaptation)**.

Em vez de atualizar todos os parâmetros do modelo original, o LoRA adiciona pequenas matrizes treináveis às camadas selecionadas, mantendo a maior parte dos parâmetros originais congelada.

Os resultados da configuração utilizada foram:

Indicador	Resultado
Parâmetros totais	1.548.072.448
Parâmetros treináveis	4.358.144
Percentual treinável	0,2815%

Dessa forma, aproximadamente **99,72% dos parâmetros permaneceram congelados** durante o treinamento.

Essa abordagem permitiu reduzir a quantidade de parâmetros efetivamente atualizados e tornou o processo mais adequado ao ambiente computacional disponível.

5. Desafios técnicos e decisões de implementação

5.1 *max_length*

Foi necessário definir um limite adequado para o número de tokens processados.

O código está puxando de forma automática o parâmetro da configuração do modelo, sendo necessário o ajuste manual.

Valores muito baixos poderiam truncar informações relevantes dos textos médicos, enquanto valores elevados aumentariam o consumo de memória.

A definição do `max_length` buscou equilibrar a preservação do contexto e a viabilidade computacional do treinamento e da inferência.

5.2 *temperature*

Também foi necessário controlar o parâmetro `temperature` durante a geração das respostas.

Como o sistema possui finalidade médica e utiliza informações recuperadas pelo RAG, buscou-se uma geração mais consistente e menos aleatória, evitando respostas excessivamente criativas.

A temperatura, entretanto, não garante a correção das respostas por si só. Ela atua principalmente sobre o grau de aleatoriedade da geração.

5.3 *Memória da GPU*

O tamanho do modelo tornou o gerenciamento da memória da GPU uma questão importante durante o treinamento e a inferência.

A utilização de LoRA reduziu significativamente a quantidade de parâmetros treináveis, mas o consumo de memória também depende de fatores como:

- tamanho dos textos;
- `max_length`;
- quantidade de dados processados;

- tamanho do contexto;
- documentos recuperados pelo RAG.

Durante o desenvolvimento, foi necessário considerar esses fatores para manter o processamento dentro dos recursos disponíveis.

5.4 Contexto do RAG

A utilização do RAG acrescentou uma nova dimensão ao gerenciamento de memória.

Os documentos recuperados pelo banco vetorial são incorporados ao prompt enviado ao modelo. Consequentemente, quanto maior o contexto recuperado, maior a quantidade de tokens processados durante a inferência.

Assim, a quantidade de documentos recuperados precisa ser equilibrada entre a disponibilidade de informação e o custo computacional.

6. Assistente médico desenvolvido

O assistente foi desenvolvido para receber uma pergunta médica, processá-la, recuperar informações relacionadas na base MedQuAD e utilizar esse contexto para gerar uma resposta.

O sistema combina o conhecimento adquirido durante a adaptação do modelo com conhecimento externo recuperado dinamicamente por meio do RAG.

Essa combinação permite que o modelo utilize uma base específica durante a geração da resposta.

O sistema foi estruturado como **protótipo de apoio à informação médica** e não deve ser interpretado como substituto da avaliação ou decisão de profissionais de saúde.

7. RAG com LangChain e Chroma

Embora o RAG não esteja apresentado de forma explícita como requisito específico do exercício, ele foi incorporado ao projeto como uma **estratégia complementar ao fine-tuning**, com o objetivo de ampliar e aprofundar o conhecimento disponível para o assistente.

Enquanto o fine-tuning busca adaptar o comportamento e o conhecimento do modelo aos dados utilizados durante seu treinamento, o RAG permite consultar uma base externa no momento da pergunta.

Dessa forma, as duas abordagens atuam de maneira complementar:

- o **fine-tuning** contribui para a adaptação do modelo ao domínio;
- o **RAG** permite fornecer informações adicionais e contextualizadas durante a geração da resposta.

Os documentos do MedQuAD foram transformados em documentos adequados ao processamento pelo LangChain e utilizados para criação de representações vetoriais.

Essas representações foram armazenadas no **Chroma**, utilizado como banco de dados vetorial.

Quando uma nova pergunta é realizada, o sistema realiza uma busca por similaridade para identificar documentos relacionados à consulta.

Os documentos recuperados são então incorporados ao contexto enviado ao modelo.

O fluxo pode ser representado da seguinte forma:

Pergunta → processamento → busca vetorial → documentos relevantes → contexto → prompt → LLM → resposta

A adoção do RAG, portanto, foi uma decisão de arquitetura para **aprofundar e complementar o conhecimento do modelo**, permitindo que o assistente consulte informações específicas sem depender exclusivamente do conhecimento armazenado em seus parâmetros.

8. Fluxo do sistema com LangChain

O LangChain atua como elemento de integração entre as diferentes etapas, coordenando o processamento da pergunta, a recuperação dos documentos e o envio do contexto para o modelo.

O fluxo geral do sistema é:

Pergunta do usuário > Processamento da pergunta > Busca no Chroma — RAG > Documentos/contexto recuperados > Construção do prompt > LLM adaptada com fine-tuning > Resposta contextualizada

Essa arquitetura permite combinar o modelo de linguagem com uma fonte externa de conhecimento, evitando que toda a informação utilizada na resposta precise estar armazenada diretamente nos parâmetros do modelo.

9. Avaliação do modelo

A avaliação foi estruturada considerando tanto o modelo adaptado quanto o comportamento do sistema RAG.

Entre os aspectos relevantes estão:

- capacidade de recuperar documentos relevantes;
- aderência da resposta ao contexto recuperado;
- relevância da resposta em relação à pergunta;
- consistência das respostas;
- adequação da resposta ao domínio médico.

Para as respostas geradas, podem ser consideradas características como:

- relevância;
- fidelidade ao contexto;
- adequação à pergunta;
- ausência de informações não sustentadas pelos documentos recuperados.

Também são relevantes métricas operacionais, como tempo de resposta e ocorrência de respostas inadequadas, estas não foram tratadas, mas caberia para uma nova versão.

Os resultados quantitativos dessas métricas não são apresentados como valores definitivos quando não foram efetivamente medidos no pipeline desenvolvido.

10. Análise dos resultados

Os resultados obtidos demonstram a viabilidade da integração entre **fine-tuning, LoRA, LangChain, Chroma e RAG** em um ambiente de recursos computacionais limitados.

Um dos principais resultados técnicos foi a redução expressiva da quantidade de parâmetros treináveis por meio do LoRA: apenas **0,2815% dos parâmetros totais do modelo foram atualizados**.

A utilização do MedQuAD como base de recuperação permitiu estruturar uma camada de conhecimento externo ao modelo.

Isso é relevante no domínio médico, no qual a resposta pode depender de informações específicas presentes em uma fonte de referência.

Por outro lado, o desenvolvimento evidenciou desafios relacionados ao gerenciamento de contexto, quantidade de tokens, memória da GPU e parâmetros de geração.

O resultado deve, portanto, ser analisado pela integração entre modelo, recuperação de conhecimento, contexto e controle da geração.

11. Segurança e limitações

Por se tratar de um domínio médico, o sistema deve ser utilizado como ferramenta de apoio à informação e não como substituto da decisão de profissionais de saúde.

Uma aplicação de produção deveria incorporar mecanismos adicionais de segurança, incluindo:

- validação humana das respostas;
- controle de acesso;
- rastreabilidade das informações utilizadas;
- identificação das fontes recuperadas;
- mecanismos de auditoria;
- tratamento adequado de dados sensíveis.

O projeto utiliza **bases públicas como alternativa acadêmica aos dados internos de um hospital**.

Portanto, não deve ser interpretado como implementação de um sistema treinado com dados clínicos internos reais.

11.1 Logging

A funcionalidade de **logging não foi incluída no protótipo desenvolvido**.

Sua implementação é considerada uma melhoria necessária para uma versão futura do assistente.

12. Considerações finais

O projeto demonstrou a construção de um protótipo de assistente médico baseado em LLM utilizando uma arquitetura que combina **fine-tuning, LoRA, RAG, LangChain e Chroma**.

O PubMedQA foi utilizado como base para a preparação e adaptação do modelo, enquanto o MedQuAD foi utilizado como fonte de conhecimento para o mecanismo de recuperação.

A utilização do LoRA permitiu reduzir significativamente a quantidade de parâmetros treináveis, tornando o processo mais adequado ao ambiente computacional utilizado.

O código final constitui um **protótipo acadêmico de assistente médico baseado em LLM e RAG**, demonstrando a integração das principais tecnologias propostas no desafio.